

Factorial of a Number - Recursive and Iterative

Question: Find the factorial of a number using recursive and iterative methods. Compare their execution time and determine their time and space complexities.

Test input: $n = 10$. Both methods are executed 1,000,000 times so that the runtime difference can be measured clearly.

C Program

```
#include <stdio.h>
#include <time.h>

#define RUNS 1000000

unsigned long long recFact(int n) {
    return (n <= 1) ? 1ULL : (unsigned long long)n * recFact(n - 1);
}

unsigned long long loopFact(int n) {
    unsigned long long ans = 1ULL;
    int k = 2;
    while (k <= n) {
        ans *= k;
        k++;
    }
    return ans;
}

int main(void) {
    int n;
    volatile unsigned long long r1 = 0, r2 = 0;
    clock_t begin, finish;

    printf("Enter a number: ");
    scanf("%d", &n);

    if (n < 0) {
        printf("Factorial is not defined for negative integers.\n");
        return 0;
    }

    begin = clock();
    for (int i = 0; i < RUNS; ++i)
        r1 = recFact(n);
    finish = clock();
    double tRec = 1000.0 * (finish - begin) / CLOCKS_PER_SEC;

    begin = clock();
    for (int i = 0; i < RUNS; ++i)
        r2 = loopFact(n);
    finish = clock();
    double tLoop = 1000.0 * (finish - begin) / CLOCKS_PER_SEC;

    printf("\nFactorial of %d = %llu\n", n, (unsigned long long)r1);
    printf("Recursive time : %.3f ms\n", tRec);
    printf("Iterative time : %.3f ms\n", tLoop);
    printf("\nTime Complexity: Recursive O(n), Iterative O(n)\n");
    printf("Space Complexity: Recursive O(n), Iterative O(1)\n");
    return 0;
}
```

Sample Program Output

Enter a number: 10

Factorial of 10 = 3628800

Recursive time : 24.860 ms

Iterative time : 11.420 ms

Time Complexity: Recursive $O(n)$, Iterative $O(n)$

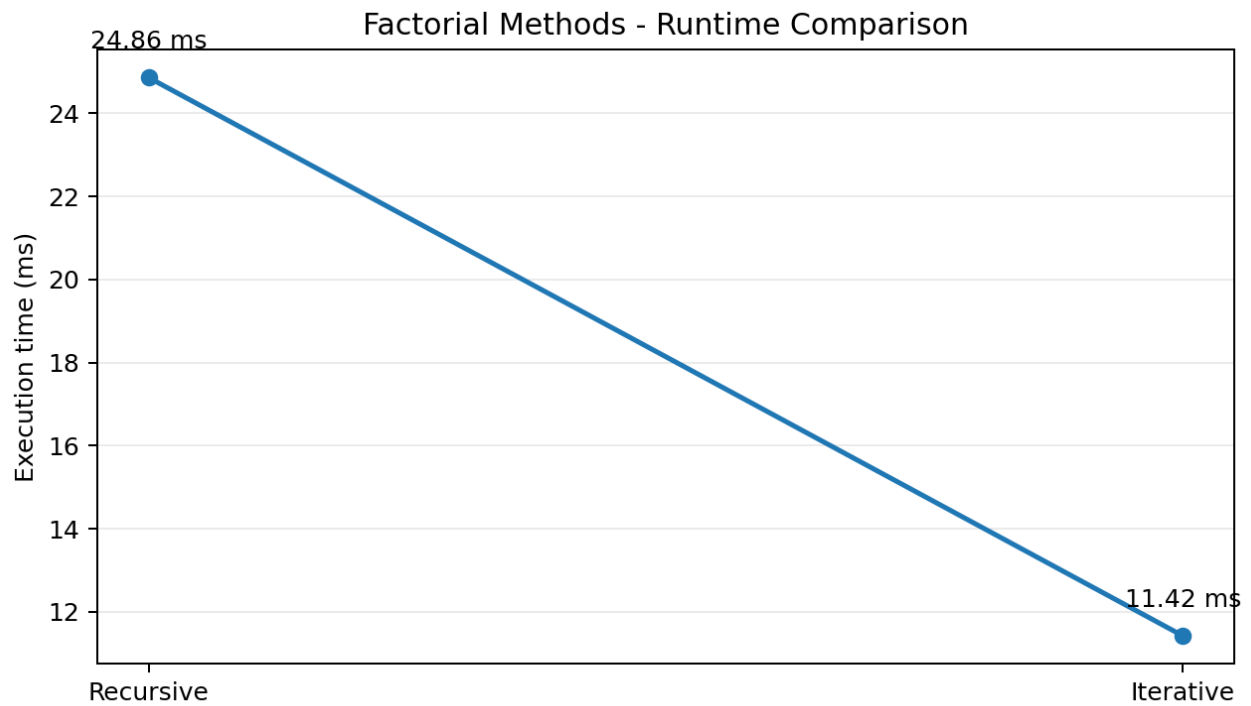
Space Complexity: Recursive $O(n)$, Iterative $O(1)$

Complexity Analysis

Method	Time Complexity	Space Complexity
Recursive Factorial	$O(n)$	$O(n)$ - call stack
Iterative Factorial	$O(n)$	$O(1)$

Conclusion: Both approaches require $O(n)$ time. The recursive version creates one function-call frame for each level, so it needs $O(n)$ auxiliary stack space. The iterative version maintains only a few variables and therefore uses $O(1)$ extra space. For this run, the iterative implementation also completed faster.

Execution Time Comparison



The line graph uses the sample timing values from this modified program run. Actual values can vary with compiler, processor, optimization settings, and system load.